

COM1025: Web and Database Systems

The HTTP and Further JavaScript

(Slides credited to Dr. Manos Panaousis)

Week 8

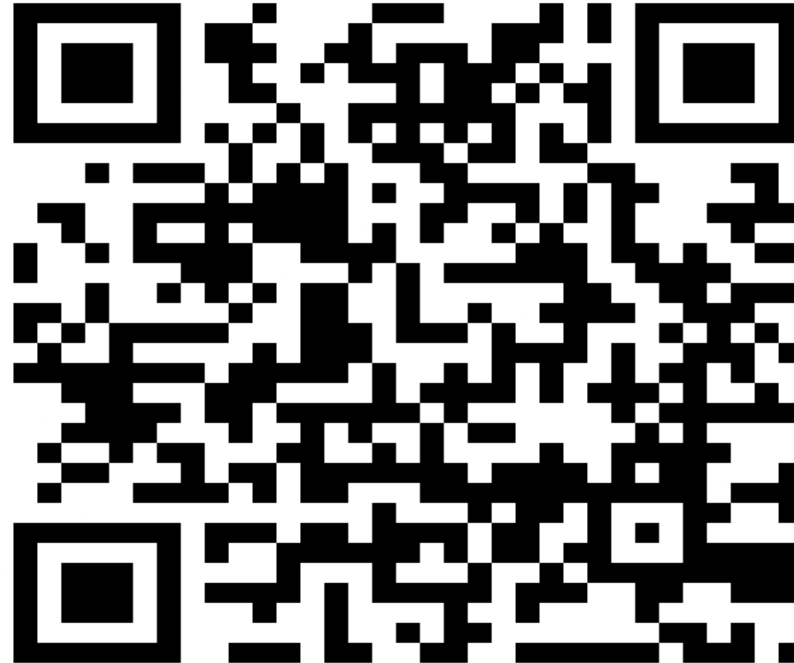
LAST WEEK

- We considered the foundations of how the internet works
- We created some simple JavaScript programs. We ran these programs in the Node.js environment.
 - However, we did not use any Node.js features. We just used JavaScript features.

THIS WEEK

- This week, we will be learning about Node.js features, and why we need them
- Specifically, we will consider:
 - ✓ How to create simple web applications
 - ✓ Using templating languages
 - ✓ Using a package manager
 - ✓ Using a **web** framework

HTTP



Visit: <https://http-server-leqm.onrender.com/>

- The Hypertext Transfer Protocol (HTTP) is designed to enable communications between clients and servers.
- HTTP is a **two-party communication** protocol:
 - ✓ Party 1 – The **client** @ Computer 1: A **web browser** (also called a “user agent”) that requests web pages
 - ✓ Party 2 – The **server** @ Computer 2: A program that handles requests of web pages from multiple clients
- Steps:
 - ✓ A client submits an **HTTP request** to the server.
 - ✓ The server returns an **HTTP response** to the client.
 - ✓ The response contains **status information** about the request and may also contain the **requested content**.
- HTTP uses the Transmission Control Protocol (TCP)

Web browser vs. Web (HTTP) server

- The program running at the server side is normally called a **web server** or an **HTTP server**

- ✓ Two most widely used web (HTTP) servers

- ❖ Apache HTTP Server: **cross-platform**, used in our labs and for your coursework Microsoft IIS (Internet Information Services, formerly known as Internet Information Server): Windows platforms only

- ❖ **NodeJS** (**We will use this one**)

- **Security?**

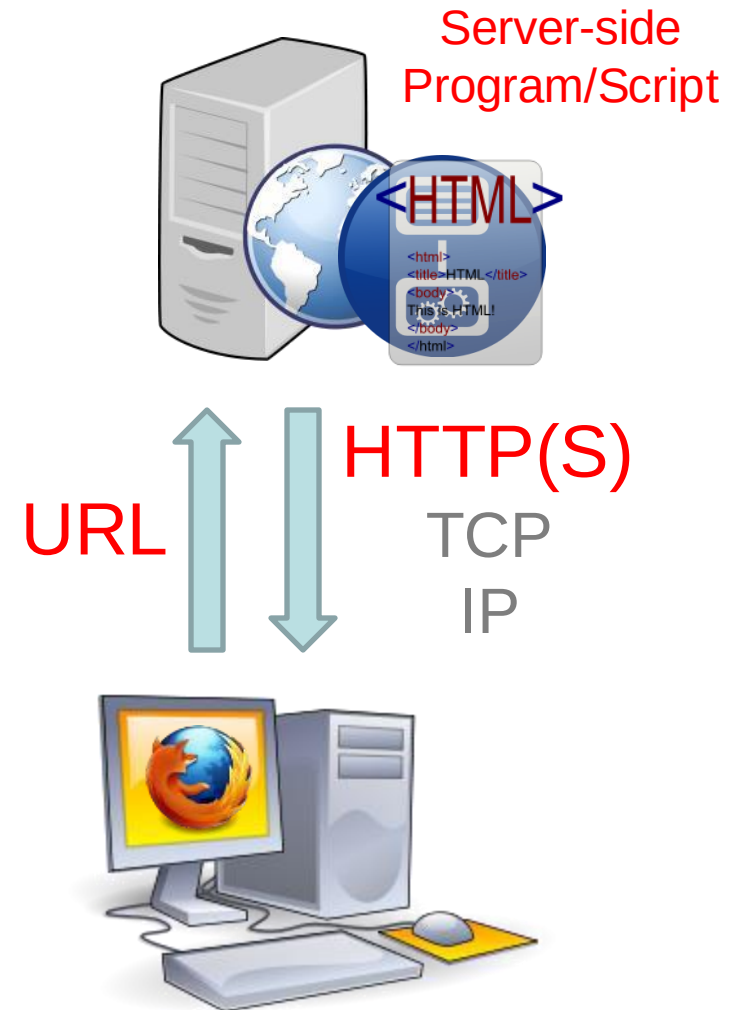
- It has a secure edition **HTTPS (HTTP Secure)**

- ✓ HTTPS allows both **end-to-end encryption** of all communications between the web server and the web browser, and also server/client authentication
- ✓ It uses a cryptographic protocol called TLS/SSL



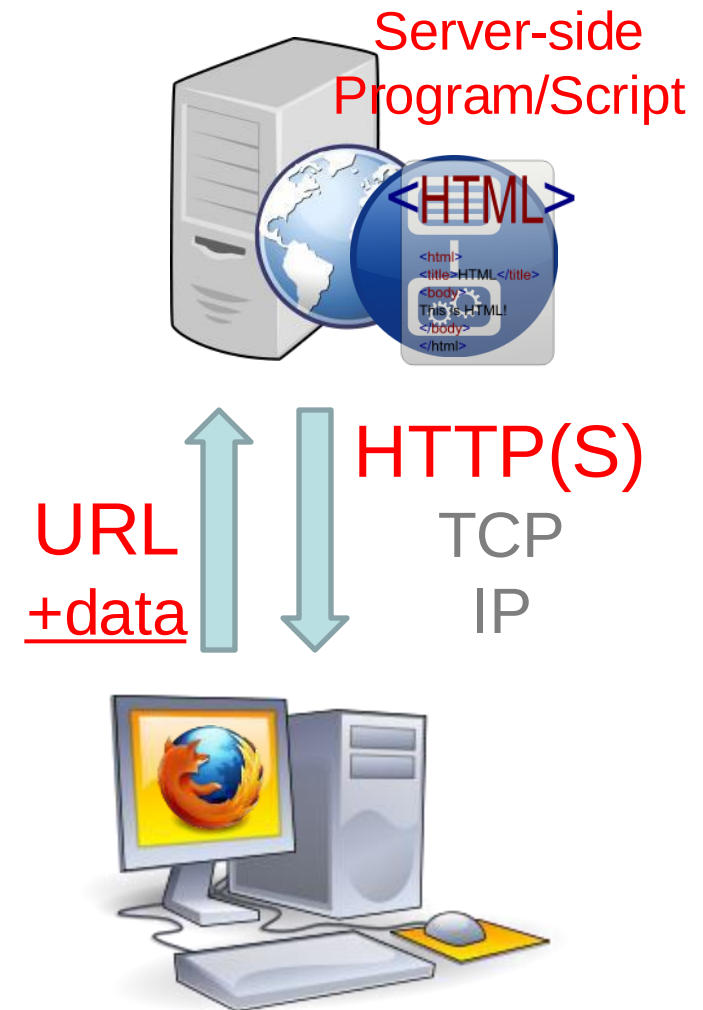
From static to dynamic web pages

- A requested web page is not necessarily a static one
- It can be a **server-side program/script** residing on the web server
- Why do we need dynamic programs/scripts?
 - ✓ Handle dynamic data.
 - ✓ Interact with users.
 - ✓ Do more with less code!
 - ✓ ...



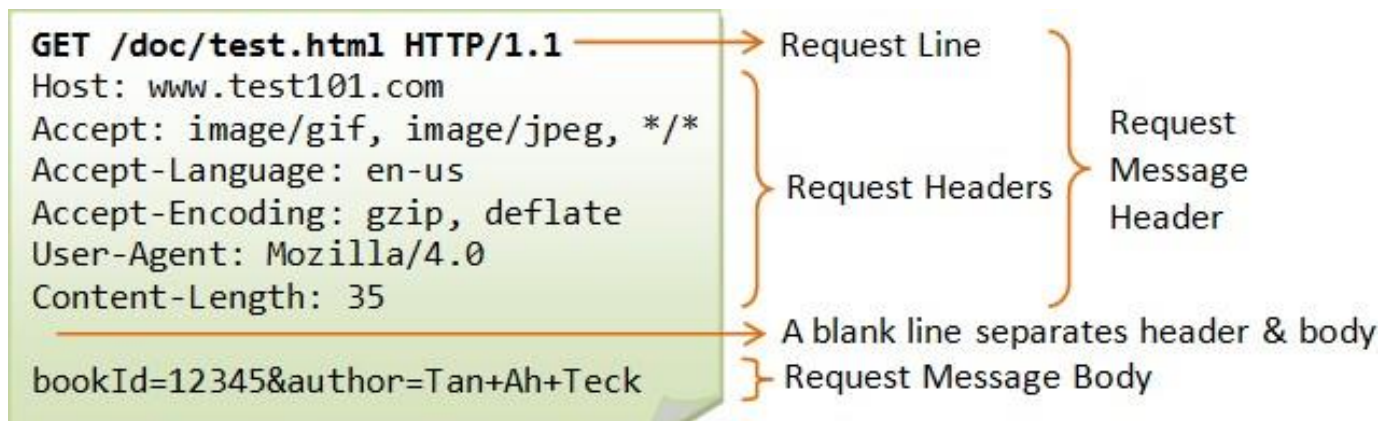
Request a dynamic web page

- User $\Rightarrow$ Web browser
 - ✓ URL <http://...> (**optionally + data**)
- Web browser $\Rightarrow$ Web server
 - ✓ URL <http://...> (**optionally + data**)
- Web server:
 - ✓ URL $\Rightarrow$ Local file path
 - ✓ Execute the local file as a program and record the output (HTML code)
- Web server $\Rightarrow$ Web browser
 - ✓ HTML code generated above
- Web browser $\Rightarrow$ User
 - ✓ The received HTML code shown (in a web browser) as a web page



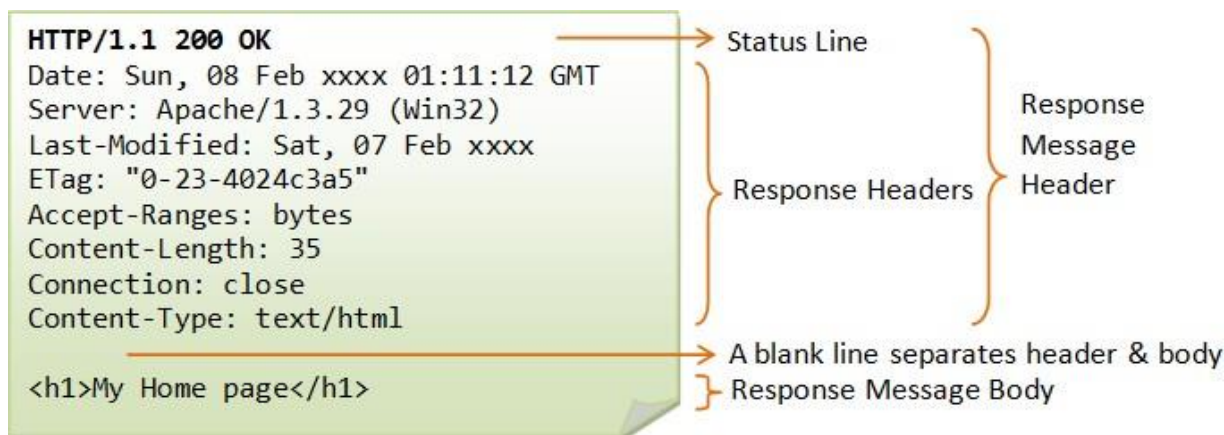
Sending a client request

- Once the connection is established, the **user-agent** (e.g. web browser, a crawler) can send the request
- A **client request** consists of **text directives** divided into:
 - ✓ The **first line** contains a **request method** (GET, POST) followed by its parameters:
 - ❖ **the path of the document**, i.e. an **absolute URL** without the protocol or domain name
 - ❖ **the HTTP protocol version**



Structure of a server response

- The web server processes the request, and ultimately **returns a response**
- A server response is formed of text directives, divided into the following blocks:
 - ✓ The first line, the status line, consists of an **acknowledgment of the HTTP version used**, followed by a **status request** (and its brief meaning in human-readable text).
 - ✓ Subsequent lines represent specific **HTTP headers**, giving the client information about the data sent (e.g. type, data size, compression algorithm used, hints about caching).



The GET method

- the **query string** (name/value pairs) is sent **in the URL** of a GET request:
 - ✓ e.g., **/test/demo_form.php?name1=value1&name2=value2**
 - ❖ GET method sends the **encoded user information** appended to the page request
 - ❖ The page and the encoded information are separated by the **?** Character
- The GET method produces a long string that **appears in your server logs**
- GET requests should be used only to retrieve data
 - ✓ GET requests should **never** be **used** when dealing with **sensitive data**
- Note: GET **can't** be used to send binary data, like images or word documents, to the server (string exclusively)
- To be covered: PHP provides **\$_GET associative array** to access all the sent information using GET method
- GET requests remain in the browser history and they **can be bookmarked**

The POST method

- the **query string** (name/value pairs) is sent **in the HTTP message body** of a POST request:

```
POST / HTTP/1.1
Host: foo.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 13

say=Hi&to=Mom
```

- It has no limit on data size (unlike GET method that is capped by max-length of URL)
- The **POST method transfers information** via HTTP so its security depends on HTTP protocol:
 - ✓ By using Secure HTTP you can make sure that your information is secure
- To be covered:** PHP provides **\$_POST associative array** to access all the sent information using the POST method
- POST requests do not remain in the browser history and they **cannot be bookmarked**

Response status codes

3. Redirection messages:

- ❖ e.g. [301 Moved Permanently](#): indicates that the resource requested has been definitively moved to the URL given by the **Location** headers

```
1 GET /index.php HTTP/1.1
2 Host: www.example.org
```

Server response 

```
1 HTTP/1.1 301 Moved Permanently
2 Location: http://www.example.org/index.asp
```

4. Client error responses:

- ❖ e.g. [404 Not Found](#): the server can not find requested resource
 - This response code is probably the most famous one due to its frequent occurrence on the web
- ❖ e.g. [403 Forbidden](#): indicates that the server understood the request but refuses to authorise it

5. Server error responses:

- ✓ e.g. 500 Internal Server Error
 - ❖ Wrong implementation of server side script



The Web/HTTP is stateless

- HTTP protocol itself is **stateless** (**memoryless**)
 - ✓ So is the Web!
 - ✓ Each HTTP request is processed without knowledge of any prior requests
 - ✓ A stateless protocol **does not** require the server to **retain information** or status about each user for the duration of multiple requests
- **No need to dynamically allocate storage** to deal with conversations in progress
- It **minimises time/bandwidth** that would otherwise be spent reestablishing a connection for each request
- If a client session dies in mid-transaction, no part of the system needs to be responsible for cleaning up the present state of the server



The Web/HTTP is stateless

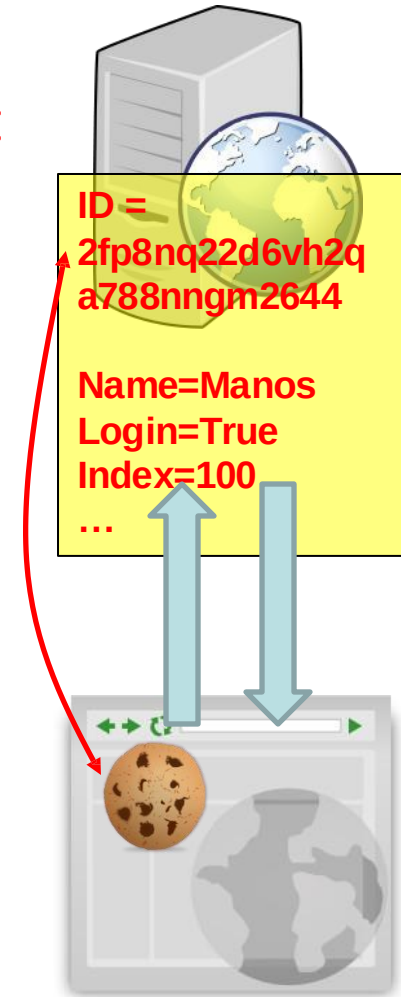
- In many applications, we want the web server to have memory of the client's visiting **history**
 - ✓ E.g., For a registered user of a website, he/she should be able to log in once for the whole website
- So we need some mechanism to make the Web (HTTP) **stateful** (= **memoryful**).
 - ✓ Save information about **historical visits** to the web server
 - ❖ So the web server keeps a memory of the past
- What **information** do we need **to save**?
 - ✓ Data submitted via HTML forms
 - ✓ Data submitted via URLs (query strings, may not be via a form)
 - ✓ **Who** visited **what**, **when** and **from where** and **how**?
 - ❖ Maximally, everything about the visit history of every visitor!



Making Web/HTTP stateful

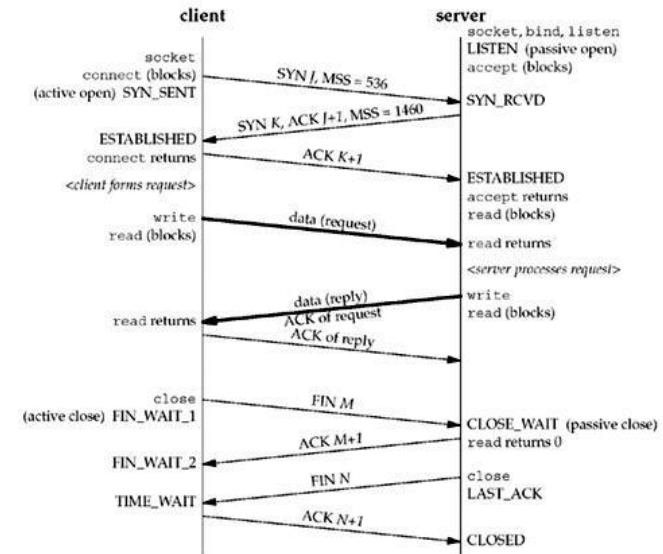
- Q: How do we save such information?
- A: Multiple ways
 - ✓ As **web (HTTP) cookies** (client-side data, short- and long-term)
 - ✓ As **session variables** (server-side data, short-term)
 - ✓ In **databases** (server-side, long-term)
 - ✓ In **data files** (server-side data, long-term)
- We will cover everything!

- A (client-side) **Web/HTTP session** lasts **from the first time the user visits a web server until the web browser closes**
- Q: How is the **session ID** maintained during a Web session?
 - ✓ Stored as a (client-side) **session cookie**
 - ✓ **Communicated as a URL parameter** as part of the query string of each URL
- Note: Each **client-side session** has a **unique ID** and can be associated with a **number of session variables** stored at the **server side**



A typical HTTP session (3 phases)

1. The client **establishes** a TCP connection:
 - ✓ The client must know the **IP address** and the **port number** of the desired server
2. The client sends its request to the server, and waits for the answer
3. The server processes the request, sending back its answer, providing a **status code** and returned data

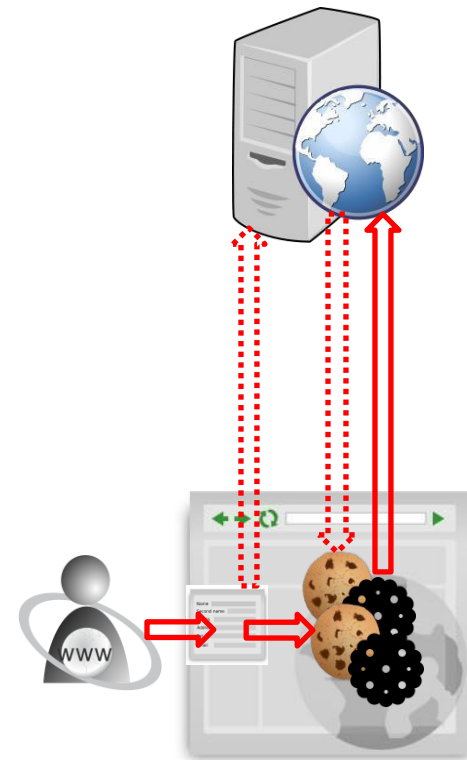


Code	Description	Code	Description
200	OK	400	Bad Request
201	Created	401	Unauthorized
202	Accepted	403	Forbidden
301	Moved Permanently	404	Not Found
303	See Other	410	Gone
304	Not Modified	500	Internal Server Error

Cookies

HTTP Cookies

- An HTTP cookie is a **small piece of text file** that is downloaded onto 'terminal equipment' after sent by the server to the user's web browser when the user accesses a website:
 - ✓ It remembers stateful information for the stateless HTTP protocol
- It allows the website to **recognise the user's device** and store some information about the user's **preferences** or **past actions**
- When receiving an HTTP request, a server can send a Set-Cookie header with the HTTP response
 - ✓ Set-Cookie: <cookie-name>=<cookie-value>



HTTP cookies

- Cookies can be used for a variety of reasons:
 - ✓ **PERSONALISATION** - **remember your preferences** on a site:
 - ❖ e.g. whether you read the oldest or newest comments first; the volume on the video player.
 - ✓ **TRACKING** - **understand how** you are **using** the site:
 - ❖ e.g. to tell what the most popular news story of the day is; to record how you responded to a new design or version of the site.
 - ✓ **SESSION MANAGEMENT** - for **logging in to a service** or to make sure you're **logged in securely**:
 - ❖ these cookies may contain information such as your email address and your name – the information you gave when you signed up
 - ❖ the website you signed up to **is the only site** that can access this information



(((BREAKING NEWS)))



Username

Password

☐ Remember Me





Cookies are sent with every request, so they can **worsen performance** (especially for mobile data connections)

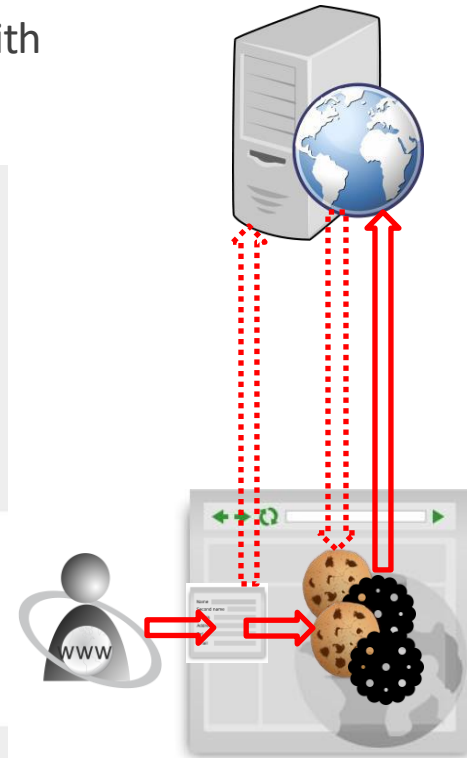
Creating cookies

- Cookie is composed by server-side program/script (e.g., PHP) and sent to the client via “Set-Cookie” in HTTP response
- The cookie can be [stored by the browser](#), and then the cookie is sent with requests made to the same server inside a [Cookie](#) HTTP header

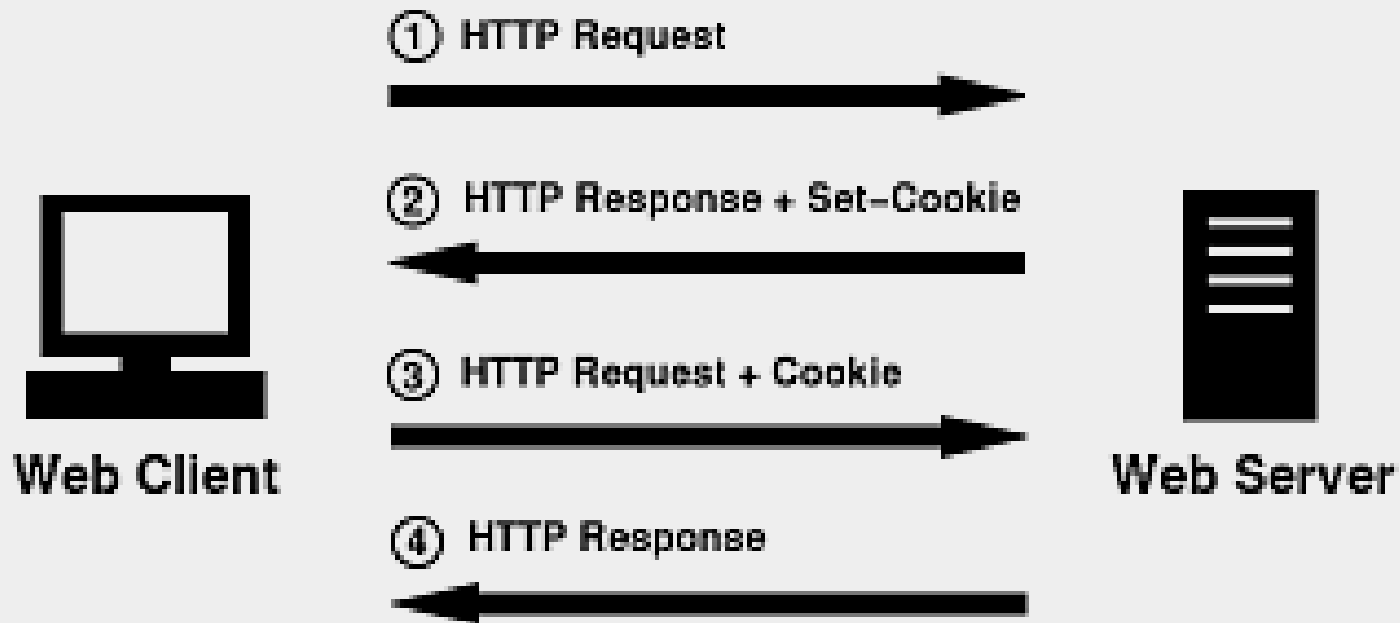
```
1 HTTP/1.0 200 OK
2 Content-type: text/html
3 Set-Cookie: yummy_cookie=choco
4 Set-Cookie: tasty_cookie=strawberry
5
6 [page content]
```

Now, with every new request to the server, the browser will send back all previously stored cookies to the server using the [Cookie](#) header.

```
1 GET /sample_page.html HTTP/1.1
2 Host: www.example.org
3 Cookie: yummy_cookie=choco; tasty_cookie=strawberry
```



HTTP cookies



Session and Permanent cookies

- The cookie created above is a *session cookie*
 - ✓ it is deleted when the client shuts down, because it didn't specify an Expires or Max-Age directive.
 - ✓ However, web browsers may use **session restoring**, which makes most session cookies permanent, as if the browser was never closed.
- Instead of expiring when the client closes, *permanent cookies* expire at a specific date (Expires) or after a specific length of time (Max-Age).

```
1 | Set-Cookie: id=a3fWa; Expires=Wed, 21 Oct 2015 07:28:00 GMT;
```

📌 **Note:** When an expiry date is set, the time and date set is relative to the client the cookie is being set on, not the server.

- Cookies are stored for **a particular domain**:
 - ✓ They can be made accessible to a single domain name (e.g. www.foo.com) or all its subdomain names (e.g. *.foo.com).
- Cookies have **an expiration time**:
 - ✓ Default: expire once the web browser closes = **session cookies**
- Secure Cookies (*): Cookies made valid for HTTPS only
 - ✓ A secure cookie will only be sent to the server when a request is made using SSL and the HTTPS protocol
 - ✓ However, **confidential or sensitive information should never be stored or transmitted in HTTP Cookies** as the entire mechanism is inherently insecure and this doesn't mean that any information is encrypted



HttpOnly cookies

- HttpOnly Cookies (*):
 - ✓ Cookies available to server only
 - ✓ HttpOnly is an **additional flag** included in a **Set-Cookie HTTP response header**
 - ✓ Using the HttpOnly flag when generating a cookie helps mitigate the risk of client side script accessing the protected cookie.
 - ❖ To prevent cross-site scripting ([XSS](#)) attacks, HttpOnly cookies are inaccessible to JavaScript's [Document.cookie](#) API



```
Set-Cookie: id=a3fWa; Expires=Wed, 21 Oct 2015 07:28:00 GMT; Secure; HttpOnly
```

Privacy?

- Contrary to a common belief, **cookies do not contain software programs**, so cannot install anything on a computer
- Cookies generally **do not contain any information** that would identify a person (**Personal Identifiable Information - PII**)
 - ✓ Usually they contain a string of text or "unique identifier" acting like a label
 - ✓ When a website sees the string of text it set in a cookie, it knows the browser is one it has seen before
- However, cookies **may violate the user's privacy** as it gives the remote server **access to local data**
- Cookies **can be disabled** in the web browser by the user



The GDPR, Cookie Consent

- The EU General Data Protection Regulation (**GDPR**) has now been in force
- The **GDPR** is an over-arching piece of legislation dealing with all aspects of the processing of personal information
- Cookies are mentioned once in the **GDPR**, in **Recital 30**:
 - ✓ *“Natural persons may be associated with online identifiers...such as internet protocol addresses, **cookie identifiers** or other identifiers.... This may leave traces which, in particular when combined with unique identifiers and other information received by the servers, may be used to create profiles of the natural persons and identify them.”*

THE ANATOMY OF A NODE PROJECT



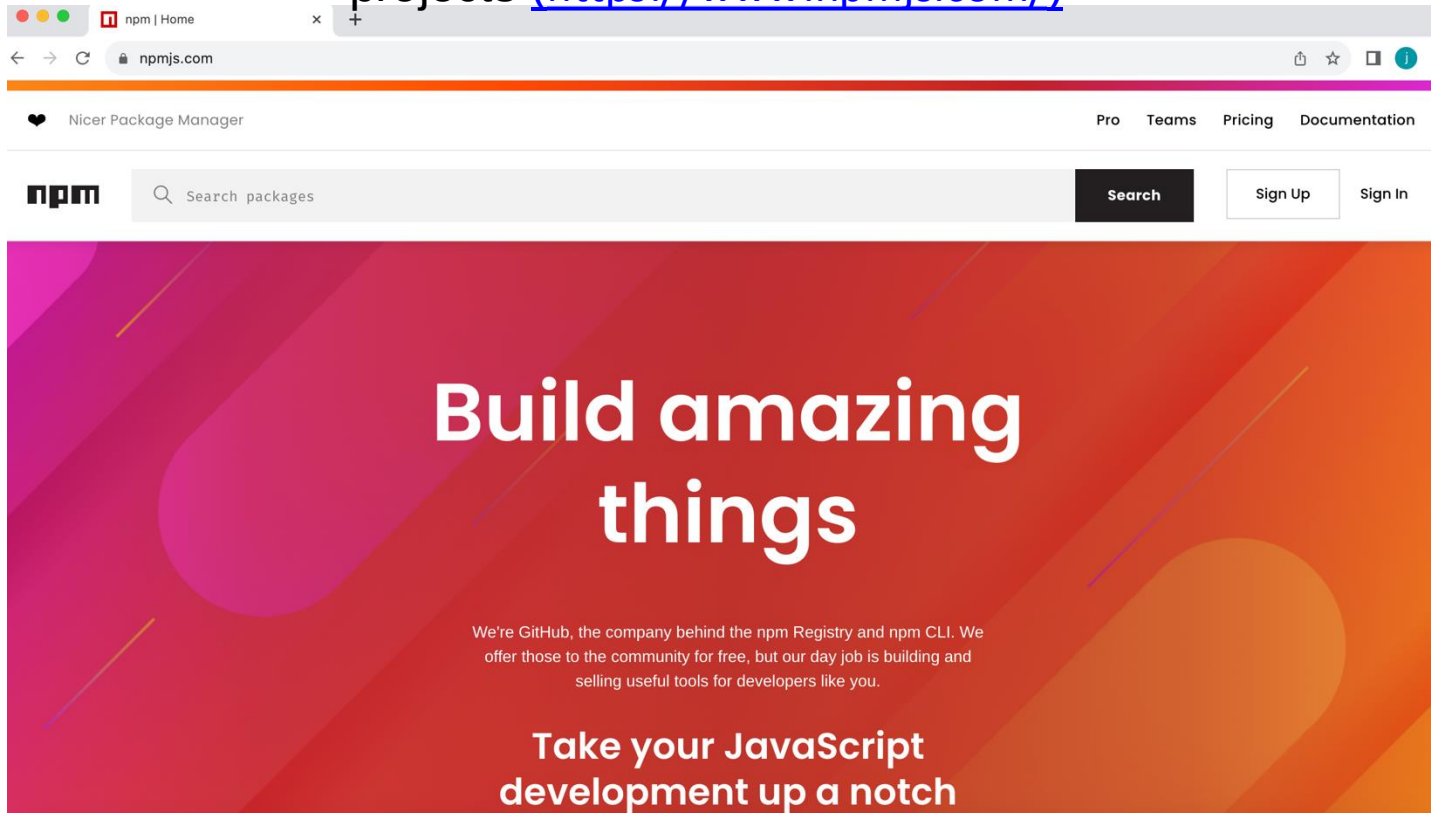
A TYPICAL NODE.JS PROJECT

```
| — my-project  
| — package.json  
| — package-  
lock.json  
| — node_modules
```

- A node project is a folder that contains a **package.json** file.
- The **package.json** file contains information about the project, including the packages it uses.
- A **package.lock** file contains information about the packages that are installed in the project
- There should be at least one JavaScript that will be the entry point for your application: **e.g., index.js, app.js**

THE NODE PACKAGE MANAGER

The node package manager can be used to manage node projects [\(https://www.npmjs.com/\)](https://www.npmjs.com/)



THE NODE PACKAGE MANAGER(NPM)

npm is:

- a package manager for JavaScript
- if you have Node.js installed, you have npm installed!
- the world's largest software registry
- it provides a command-line interface (CLI) for managing packages and other tasks
- it can be used to install, manage, and publish packages, manage dependencies, and run scripts

GETTING STARTED WITH NPM

- Ensure you have NodeJS installed
- In the directory of your project run the terminal command: **npm init**



Terminalizer

```
The default interactive shell is now zsh.  
To update your account to use zsh, please run `chsh -s /bin/zsh`.  
For more details, please visit https://support.apple.com/kb/HT208050.  
R2DJ2G5GWM:my_first_website ja0054$  
The default interactive shell is now zsh.  
To update your account to use zsh, please run `chsh -s /bin/zsh`.  
For more details, please visit https://support.apple.com/kb/HT208050.  
R2DJ2G5GWM:my_first_website ja0054$
```

YOU NOW HAVE A PACKAGE.JSON FILE

```
"name": "my_first_website",  
"version": "1.0.0",  
"main": "index.js",  
"scripts": {  
  "test": "echo `Error: no test  
specified`" && exit 1"  
},  
"author": "",  
"license": "ISC",  
"dependencies": {  
},  
"devDependencies": {},  
"description": ""
```

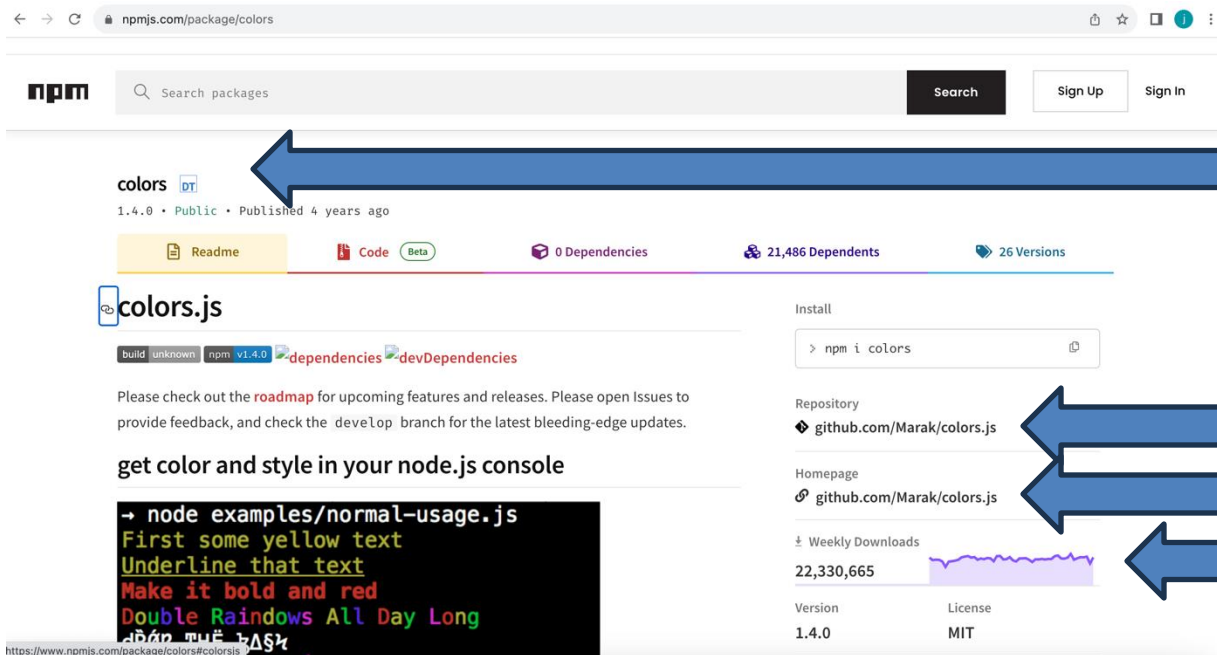
- The `package.json` file contains information about the project and is split into sections.
- For this module, there are only three sections we care about:
- The `devDependencies` field is a list of packages that the project depends on in development mode.
- The `dependencies` field is a list of packages that the project depends on.

GETTING STARTED WITH NPM

- After running “npm init”; we can now install some packages
- You can find packages by searching the <https://www.npmjs.com/> Repository
- Packages are open source and free
- There are over 1.2million packages
- Many are downloaded millions of times a week

LET'S INSTALL COLORS (I)

This package lets us print different colors to the console



The screenshot shows the npm package page for 'colors'. A large blue arrow points from the text 'Name of the package' to the package name 'colors' in the header. Another blue arrow points from the text 'The source code' to the 'Repository' link 'github.com/Marak/colors.js'. A third blue arrow points from the text 'Documentation' to the 'Homepage' link 'github.com/Marak/colors.js'. A fourth blue arrow points from the text 'Downloads' to the 'Weekly Downloads' section, which shows 22,330,665 downloads. The package page also includes a search bar, a 'Sign Up' button, a 'Sign In' button, and a 'Readme' button. The package details section shows '1.4.0 • Public • Published 4 years ago', '0 Dependencies', '21,486 Dependents', and '26 Versions'. The 'Install' section shows the command 'npm i colors'. The 'Repository' and 'Homepage' sections both point to 'github.com/Marak/colors.js'. The 'Weekly Downloads' section shows a bar chart and the number '22,330,665'. The 'Version' and 'License' sections show '1.4.0' and 'MIT' respectively. The 'devDependencies' section shows 'dependencies' and 'devDependencies'. The 'get color and style in your node.js console' section shows a terminal output with various colors and styles.

npmjs.com/package/colors

npm Search packages Search Sign Up Sign In

colors 1.4.0 • Public • Published 4 years ago

Readme Code Beta 0 Dependencies 21,486 Dependents 26 Versions

colors.js

build unknown npm v1.4.0 dependencies devDependencies

Please check out the [roadmap](#) for upcoming features and releases. Please open Issues to provide feedback, and check the `deveLop` branch for the latest bleeding-edge updates.

get color and style in your node.js console

```
→ node examples/normal-usage.js
First some yellow text
Underline that text
Make it bold and red
Double Raindows All Day Long
rain rain rain
```

Install

```
> npm i colors
```

Repository

github.com/Marak/colors.js

Homepage

github.com/Marak/colors.js

Weekly Downloads

22,330,665

Version

1.4.0

License

MIT

Name of the package

The source code

Documentation

Downloads

LET'S INSTALL COLORS.JS (II)



- Dependencies are installed from command line.
 - Ensure your terminal window is pointed to the root of your project.

> npm install colors

- Packages are stored in the "node_modules" folder
- Dependencies listed in the "package.json" file
- Packages are stored in the "node_modules" folder



```
{
  "name": "demo_1",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \\Error: no test specified\\",
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "colors": "^1.4.0"
  }
}
```

USING PACKAGES

```
const colors = require('colors'); // save the package to a const
console.log('Hello World!'.rainbow);
```

PROBLEMS

6

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

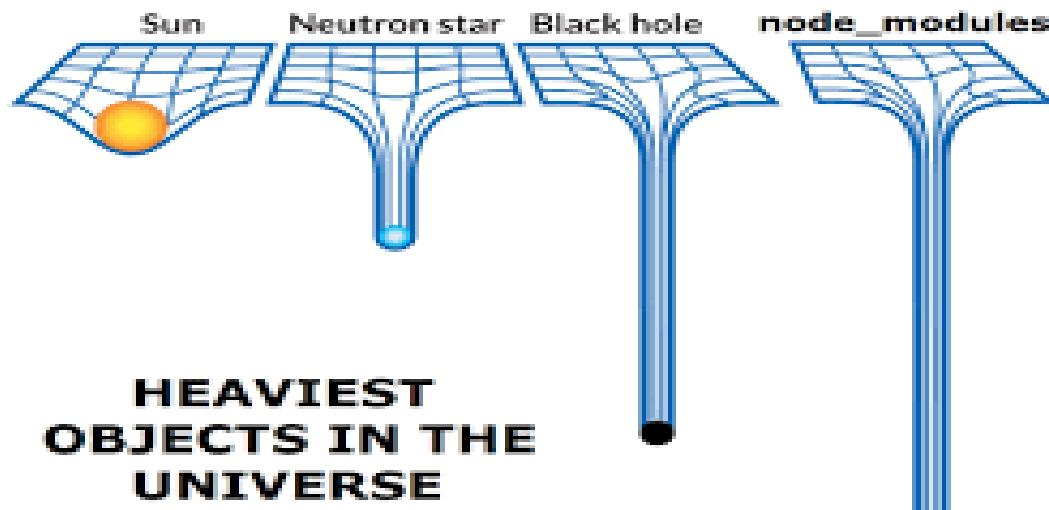
● ja0054@R2DJ2G5GWM demo_1 % node index.js

Hello World!

○ ja0054@R2DJ2G5GWM demo_1 %

SOME TIPS

- Since packages have dependencies the "**node_modules**" folder gets very heavy
- Use .gitignore to exclude "**node_modules**" from version control
- Delete the "**node_modules**" folder before submitting your assessment
- We can install the dependencies by running "**npm install**"



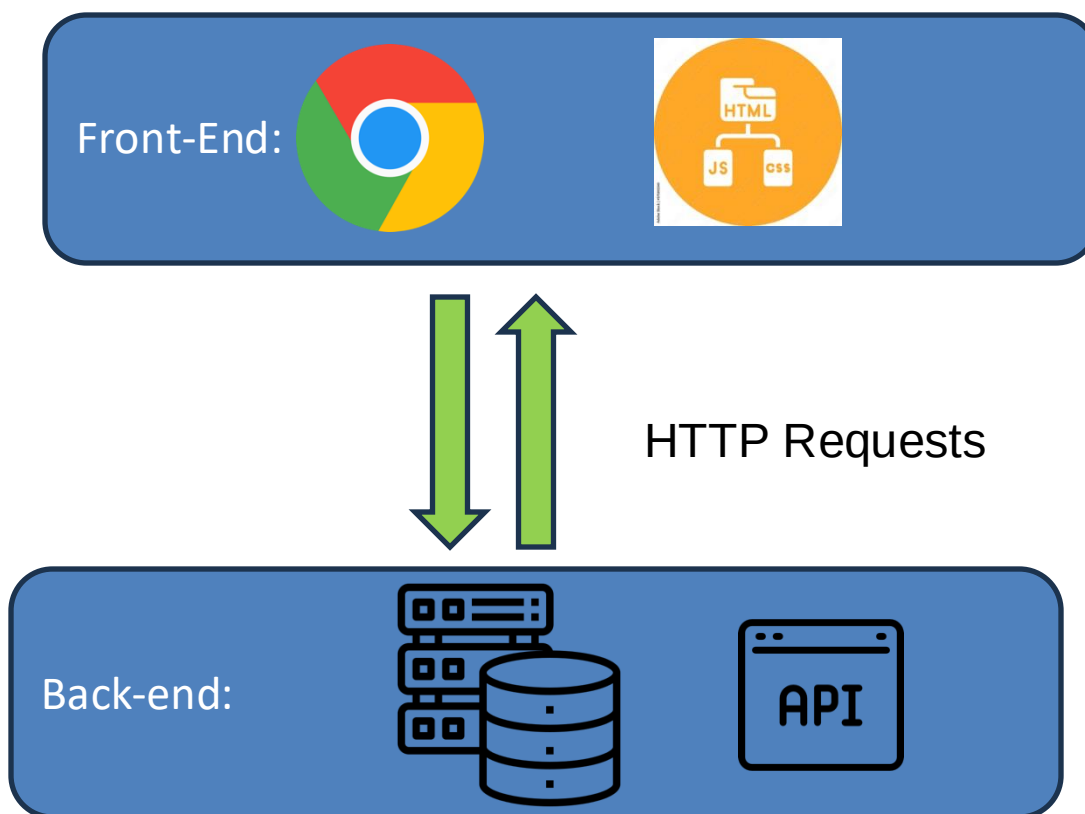
DEMO (SEE LECTURE TAB)

A series of overlapping, wavy lines in shades of yellow, orange, and blue, flowing from the bottom left towards the right side of the slide.

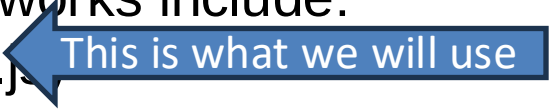
WEB DEVELOPMENT FRAMEWORKS

A series of overlapping, wavy lines in shades of yellow, orange, and blue, flowing from the bottom left towards the right side of the slide.

FRONT END VS BACK-END DEVELOPMENT



WEB FRAMEWORKS

- Web Frameworks Simplify the process of web development
- They abstract low-level details
- Common web frameworks include:
 - Express.js (Node.js)  This is what we will use
 - Django (Python)
 - Ruby on Rails (Ruby)
 - Flask (Python)
 - Laravel (PHP)
 - Spring Boot (Java)

Express 4.18.1

Fast, unopinionated, minimalist
web framework for Node.js

- ✓ A minimal and flexible Node.js web application framework
- ✓ Express includes a routing system that enables developers to define routes for different HTTP methods and URLs.
- ✓ Makes it easy to create a structured and organized API or web application with specific endpoints.
- ✓ Express can be integrated with various template engines, such as **EJS** or Pug.

CREATING AN EXPRESS APPLICATION

```
> npm install express
```

```
const express = require("express");
const app = express();
const port = 3000;

app.get("/", (req, res) => {
  res.send("Hello World!");
});

app.listen(port, () => {
  console.log(`Example app listening at http://localhost:${port}`);
});
```

- The above example uses the express package to create a new web-server that listens on port 3000. We can then set up listeners to respond to any given HTTP request.

SERVING HTML FILES

```
const express = require("express");
const path = require("path"); // this is a built in node package

const app = express();
const port = 3000;

app.get("/", (req, res) => {
  res.sendFile(path.resolve(__dirname, "index.html"));
});

app.listen(port, () => {
  console.log(`Example app listening at http://localhost:${port}`);
});
```

- Above, we return an index.html file when a request is received to the root of our website.
- path.resolve is a Node.js method that resolves a sequence of path segments into an absolute path
 - e.g., if your current directory is “/user/example” the output would be “/user/example”

USING THE STATIC MIDDLEWARE

- Currently we cannot serve static assets such as CSS and images in our HTML pages.
 - We are only sending a single HTML file, not linked assets (e.g., images, css, and JavaScript)
- To configure this
- Middleware can modules you can
- We can use mid

Express middleware

The Express middleware modules listed here are maintained by the [Expressjs team](#).

Middleware module	Description	Replaces built-in function (Express 3)
body-parser	Parse HTTP request body. See also: body , co-body , and raw-body .	<code>express.bodyParser</code>
compression	Compress HTTP responses.	<code>express.compress</code>
connect-redis	Generate unique request ID.	NA
cookie-parser	Parse cookie header and populate <code>req.cookies</code> . See also cookies and keygrip .	<code>express.cookieParser</code>
cookie-session	Establish cookie-based sessions.	<code>express.cookieSession</code>
cors	Enable cross-origin resource sharing (CORS) with various options.	NA
errorhandler	Development error-handling/debugging.	<code>express.errorHandler</code>
method-override	Override HTTP methods using header.	<code>express.methodOverride</code>
morgan	HTTP request logger.	<code>express.logger</code>
multer	Handle multi-part form data.	<code>express.bodyParser</code>
response-time	Record HTTP response time.	<code>express.responseTime</code>
serve-favicon	Serve a favicon.	<code>express.favicon</code>
serve-index	Serve directory listing for a given path.	<code>express.directory</code>
serve-static	Serve static files.	<code>express.static</code>
session	Establish server-based sessions (development only).	<code>express.session</code>
timeout	Set a timeout period for HTTP request processing.	<code>express.timeout</code>
vhost	Create virtual domains.	<code>express.vhost</code>

e middleware.
so third-party

.S

SERVING STATIC ASSETS

- Below, we instruct express to serve assets from a folder in our root directory called **public**
- **Note**, use “express.use” to set up middleware

```
const express = require("express");  
const path = require("path");  
const app = express();  
app.use(express.static(path.join(__dirname, "public")));
```

- Above, express looks up the files relative to the static directory public:
 - the name of the static directory is not
 - if we place the image foo.jpeg in the public folder we would reference it in index.html like this, .

DEMO

A series of overlapping, wavy lines in shades of yellow, orange, and blue, flowing from the bottom left towards the right side of the slide.

WHAT'S WRONG WITH JUST HTML

- Serving plain HTML files provides us with a means to present a website
- How do we inject data into our html pages?
- Shared components or layouts must be replicated across different HTML files
- As web applications grow in complexity, maintaining a large codebase written in pure HTML becomes increasingly challenging.
- The lack of a clear separation of concerns makes it difficult to manage the presentation layer independently from the application logic

TEMPLATING TO THE RESCUE

- Templating allows for the dynamic generation of content in web applications
- Templating promotes code reusability by separating the HTML structure from the data
- Shared components
- Templating ensures consistent layouts across different pages of a website.
- Some popular templating languages include: [EJS](#), [Handlebars](#), [Mustache](#) , and [Pug](#)

EXPRESS SUPPORTS TEMPLATING

- We'll use EJS
- EJS is simple to use. First, we need to install it:

```
> npm install ejs
```

- We can now tell express to render our html pages using ejs. Below is a full example.

USING EJS (I)

```
const express = require("express");
const path = require("path");
const app = express();
const port = 20000;
app.set("view engine", "ejs");
app.use(express.static(path.join(__dirname, "public")));

app.get("/", (req, res) => {
  res.render("index");
});

app.listen(port, () => {
  console.log(`Example app listening at http://localhost:${port}`);
});
```

USING EJS (II)

- Express will assume that we have an `index.ejs` file in a `views` folder which lives in the root directory of your project.
- EJS, is a superset of HTML. This means, to get the above example to work, we can simply rename our any `html` file to `index.ejs` and move it to a `views` folder.
- Using EJS, we now have some dynamic capabilities within our HTML views.

EJS ALLOWS HTML TO BE CHUNKED

```
<!-- views/common/header.ejs -->
<ul>
  <li><a href="home">Home</a></li>
  <li><a href="about"></a>About</li>
  <li><a href="contact">Contact</a></li>
</ul>
```

```
<!-- views/index.ejs -->
...
<body>
  <%- include('common/header'); %>
  <h1>Home Page</h1>
  
</body>
...
```

DIFFERENT TYPES OF TAGS

- We will investigate more dynamic EJS tags next week
- You can get a full list of tags from the EJS

Tags

- `<%` 'Scriptlet' tag, for control-flow, no output
- `<%=` 'Whitespace Slurping' Scriptlet tag, strips all whitespace before it
- `<%=` Outputs the value into the template (HTML escaped)
- `<%=` Outputs the unescaped value into the template
- `<%=#` Comment tag, no execution, no output
- `<%%` Outputs a literal '<%'
- `%>` Plain ending tag
- `->` Trim-mode ('newline slurp') tag, trims following newline
- `_>` 'Whitespace Slurping' ending tag, removes all whitespace after it

DEMO (SEE LECTURE TAB)

A series of overlapping, wavy lines in shades of yellow, orange, and blue, flowing from the bottom left towards the right side of the slide.

END OF LECTURE

A series of overlapping, wavy lines in shades of yellow, orange, and blue, flowing from the bottom left towards the right side of the slide.

End of Week 8